

Module 5 – Software Assurance Report

Pammi (JHED ID gemefie1) · Software Concepts · September 29, 2026

Grad Cafe analytics service: Pylint, SQL-injection defenses, LIMIT enforcement, least-privilege PostgreSQL role, dependency analysis, packaging, Snyk and CI.

1. Installing and running the project (pip and uv)

The project is installable: `setup.py` in `module_5/` declares the eleven modules under `src/`, reads its runtime dependencies from `requirements.txt`, and exposes console commands (`gradcafe-web`, `gradcafe-load`, `gradcafe-query`, `gradcafe-pull`). Two equivalent install paths are documented in the README and were both verified in a clean environment.

pip + venv: `python3 -m venv .venv && source .venv/bin/activate, pip install -r requirements.txt, pip install -e ., cp .env.example .env`. **uv:** `uv venv && source .venv/bin/activate, uv pip sync requirements.txt, uv pip install -e ..` `uv pip sync` makes the environment match the requirements file exactly (it removes anything not listed), which is what makes the setup reproducible; `uv pip install -e .` reads the dependency list from `setup.py`.

`requirements.txt` now contains everything needed at runtime (Flask, SQLAlchemy, psycopg, python-dotenv, beautifulsoup4, selenium, pillow), for tests (pytest, pytest-cov), for docs (sphinx, sphinx-rtd-theme) and for assurance tooling (pylint, pydeps, reportlab). After installing, `python src/flask_app.py` serves the analysis page at `http://127.0.0.1:5000/analysis` and `pytest module_5/tests -m "web or buttons or analysis or db or integration"` runs the 122-test suite at 100 % coverage.

Why packaging matters. Without a package, every entry point (terminal, pytest, CI, Sphinx) has to put `src/` on `sys.path` in its own way, and the moment two of them disagree you get "works on my machine" import errors. An editable install (`pip install -e .`) registers the source directory once, so import `flask_app` resolves identically everywhere while edits stay live; tools such as `uv` can derive the dependency set from `setup.py`; and the console-script entry points give operators stable commands instead of file paths.

2. Pylint: 10.00/10

Command (documented in the README): `pylint --rcfile=.pylintrc --fail-under=10 src`, run from `module_5/`. Output is committed as `pylint_score.txt`; the final score is 10.00/10 with no messages. Only `src/` is linted, as required.

The fixes were real changes rather than suppressions: missing docstrings added; an unused import removed; module-level mutable globals in `models.py` replaced by a small state object (no global statement); the lazy `import` statements in `scrape.py` and `pull_new_data.py` moved to module level (Selenium and Pillow are hard requirements); over-broad `except Exception` handlers narrowed to the concrete failure types (`psycopg.Error`, `SQLAlchemyError`, `RuntimeError`, `ValueError`, `OSError`); long lines wrapped; and `sqlalchemy.func.count` replaced by the `count` class so Pylint can see it is callable. `.pylintrc` sets a 120-column limit and raises four design thresholds that the application factory, the HTML parser and the resumable scraping loop legitimately exceed; no message is disabled inline.

3. SQL injection defenses (what changed and why it is safe)

Before (Module 4): `query_data.py` built its eleven statements by interpolating fragment strings into SQL text with f-strings, and `load_data.fetch_applicant()` built its column list with `', '.join(COLUMNS)`. The values were constants rather than user input, but the pattern - SQL text assembled from Python strings - is exactly what turns into an injection the first time a variable comes from a request.

After: every statement in `src/` is composed with `psycopg.sql`. The table and each column name is an `sql.Identifier`, which `psycopg` double-quotes, so a name can never be interpreted as SQL. Every value - term, status prefix, score bounds, university patterns, the applicant id, and the row limit itself - is a `%(name)s` placeholder (or `sql.Placeholder`) bound at execution time; the driver sends it as data, so `' OR 1=1 --` is compared as a literal string and matches nothing. Statement construction (module-level `sql.Composed` objects, or `build_applicants_query()`) is separated from execution (`cursor.execute(statement, params)`), and there are no f-strings, `+` or `.format()` on SQL text anywhere.

The new `GET /api/applicants` endpoint is the one place that takes caller input, so it is where the defenses are exercised end to end: `order_by` must be one of the fifteen real column names (`query_safety.safe_column()` raises `ValueError`, the route answers HTTP 400); `status` and `term` are bound parameters; `desc` is reduced to a boolean that selects between two fixed `sql.SQL` fragments; `limit` is clamped. The loader's `fetch_applicant()` converts its id with `int()` and binds it, so a hostile id simply returns `None`.

`tests/test_sql_safety.py` proves the behaviour against a real PostgreSQL: injection payloads in `status`, `term` and `order_by` (`' OR 1=1 --, UNION SELECT, p_id; DROP TABLE applicants`) return no rows or a 400, never a crash, and the row count is unchanged afterwards; the rendered SQL of every question contains quoted identifiers and no literal values; oversized, negative and non-numeric limits are clamped.

4. LIMIT enforcement

`query_safety.py` defines `MAX_LIMIT = 100`, `DEFAULT_LIMIT = 20` and `clamp_limit(value)`, which turns any input (`None`, blank, text, negative, huge) into an integer in `[1, 100]`. Every statement ends with `LIMIT %(limit)s` and passes `clamp_limit(...)` as the bound value: the eleven analysis questions (aggregates use 1, the two tables 100 and 10), the loader's count and fetch, the pipeline's known-URL lookup, the API endpoint, and every SQLAlchemy query (`.limit(clamp_limit(...))`).

The one query that used to return the whole table - the scraper's list of already-stored URLs - is now ordered newest-first and capped at 100. That is sufficient because Grad Cafe lists newest submissions first and the scraper stops at the first page containing an entry it already holds. Validation therefore prevents abuse through oversized requests at every layer: an HTTP client asking for `limit=999999999` receives 100 rows and the response reports the limit actually applied.

5. Least-privilege database configuration

Credentials. Nothing in `src/` contains a host, user or password. `config.py` reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` (or an overriding `DATABASE_URL`) from the environment, with a git-ignored `.env` loaded by `python-dotenv`; `.env.example` lists the names with placeholders. A test asserts that no credential literal appears in the source.

Role. `db/create_app_user.sql` creates `gradcafe_app` with `LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOREPLICATION CONNECTION LIMIT 5`, revokes the default `PUBLIC` privileges, and grants exactly `CONNECT` on the database, `USAGE` on schema `public` and `SELECT`, `INSERT` on `applicants`. The application reads (analysis page, API, query scripts) and, through Pull Data, inserts new rows with `ON CONFLICT DO NOTHING`; it never updates or deletes, never alters the schema, and does not own the table, so it holds none of those rights. `USAGE` without `CREATE` on the schema means it cannot create objects either. Table creation is an administrative step run once by the owner; `load_data.create_table()` now checks `to_regclass('applicants')` first and only issues `CREATE TABLE` when the table is missing, so the restricted role can run the pipeline safely.

Verification (run as `gradcafe_app` against the loaded database): `SELECT COUNT(*)` succeeds; `DELETE -> "permission denied for table applicants"`; `DROP TABLE` and `ALTER TABLE -> "must be owner of table applicants"`; `CREATE TABLE -> "permission denied for schema public"`; `pg_roles` shows `rolsuper = f`, `rolcreatedb = f`, `rolcreatorole = f`; `information_schema.role_table_grants` lists only `INSERT` and `SELECT`. The test suite uses a separate throwaway database (`gradcafe_test`) because it truncates the table before every test.

```
-- Least-privilege database account for the Grad Cafe analytics app (Module 5).
-- Run once as the database owner / superuser, e.g.:
--     psql -d gradcafe -v app_password=change-me -f db/create_app_user.sql
--
-- The application only ever:
-- * SELECTs from applicants (analysis page, /api/applicants, query scripts)
-- * INSERTs into applicants (Pull Data adds new rows; ON CONFLICT DO NOTHING)
-- It never UPDATES, DELETES, creates or alters objects, so the role gets none
-- of those rights and is explicitly NOT a superuser, cannot create databases
-- or roles, and does not own the table. Schema creation (load_data.create_table)
-- is an administrative step done by the owner before the app user is used.

\set ON_ERROR_STOP on

-- 1. Role: login only, no elevated attributes, small connection budget.
-- (Run once; re-running fails on "role already exists", which is harmless.)
CREATE ROLE gradcafe_app LOGIN PASSWORD :app_password
    NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOREPLICATION CONNECTION LIMIT 5;

-- 2. Start from nothing: remove the default PUBLIC privileges on the schema.
REVOKE ALL ON SCHEMA public FROM gradcafe_app;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM gradcafe_app;

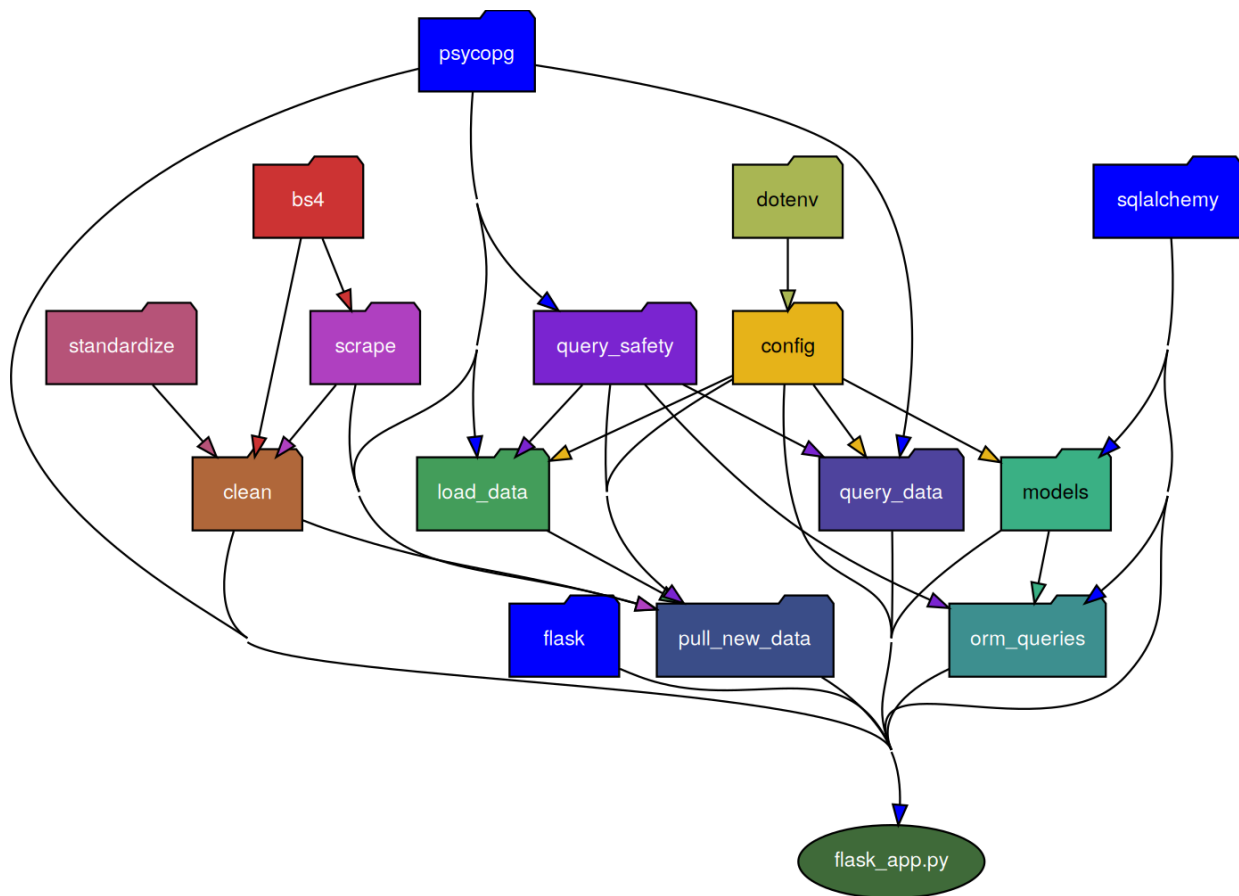
-- 3. Grant exactly what the app needs.
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;           -- see the table, cannot CREATE in the schema
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;

-- 4. (Optional) make the account read-only instead, if Pull Data is not used:
-- REVOKE INSERT ON TABLE applicants FROM gradcafe_app;

-- Verify:
-- \du gradcafe_app
-- SELECT grantee, privilege_type FROM information_schema.role_table_grants
-- WHERE grantee = 'gradcafe_app';
```

6. Dependency graph (dependency.svg)

Generated with `pydeps flask_app.py --noshow -T svg -o ../dependency.svg --max-bacon 2 --cluster from module_5/src (pydeps drives Graphviz's dot)`. The graph is rooted at `flask_app.py`, the web entry point, and shows two kinds of nodes. The project's own modules form the middle of the picture: `flask_app` imports `orm_queries` and `models` (the SQLAlchemy read path for the page), `query_data` (the composed raw-SQL path used by `/api/applicants`), `pull_new_data` (the Pull Data pipeline) and `config`; `pull_new_data` in turn pulls in `scrape`, `clean`, `load_data` and `standardize`, which is the whole ETL chain. `query_safety`, the new module, is imported by `query_data`, `load_data`, `pull_new_data` and `orm_queries` - every module that talks to the database shares the same LIMIT clamp and identifier whitelist. The external packages sit at the edges: `flask` (HTTP and templates), `sqlalchemy` (ORM, used by `models` and `orm_queries`), `psycopg` (the PostgreSQL driver, used by the raw-SQL modules and by `query_safety` for `sql.Identifier`), `bs4` (BeautifulSoup, used by `scrape` and `clean` to parse HTML) and `dotenv` (loads `.env` for config). Selenium and Pillow do not appear because `--max-bacon 2` limits the graph to two hops from the entry point and they are only reached through `scrape`. The shape - one entry point, one shared safety module, and a small set of third-party packages that each touch a well-defined layer - is the structure the CI job regenerates and validates on every push.



7. Snyc dependency scan

Dependency scan (snyk test). Run from module_5/ with the CLI authenticated to the donpams Snyk organization: `snyk test --file=requirements.txt --package-manager=pip`. Snyk resolved the installed environment (`requirements.txt`, package manager `pip`) and tested **62 dependencies** (the direct requirements plus their transitive packages); the result was **"no vulnerable paths found"** and no license issues. The full output is saved as `snyk_test_output.txt` and pictured in `snyk-analysis.png`.

Vulnerabilities and remediation. None were reported, so no package had to be patched or removed. The tip about "multiple supported manifests" refers to `setup.py` sitting next to `requirements.txt`; both describe the same dependency set (`setup.py` reads its `install_requires` from `requirements.txt`), so a `--all-projects` scan would test the same tree twice. To keep the result current the CI snyk job re-runs the same test on every push, and `snyk monitor` can be added to receive alerts when a new advisory affects one of the 62 packages.

Snyk Code (SAST, extra credit). `snyk code test` was run against module_5/; the output is captured in `snyk-code.png` / `snyk_code_output.txt`. See the note below the screenshot for the findings summary.

8. GitHub Actions CI (.github/workflows/ci.yml)

Four jobs run on every push and pull request. **pylint**: installs `requirements.txt` and runs `pylint --rcfile=.pylintrc --fail-under=10 src`, so any regression below 10.00 fails the build.

dependency-graph: installs Graphviz with `apt`, regenerates `dependency.svg` with `pydeps`, fails if the file is missing/empty or does not mention `flask_app`, and uploads it as a build artifact. **snyk**: runs `snyk/actions/python` (`snyk test` on `requirements.txt`) with the `SNYK_TOKEN` repository secret; it is `continue-on-error: true` so the scan always reports without blocking - flip that flag to block on findings.

pytest: starts a `postgres:16` service, exports `DATABASE_URL`, and runs the marked suite with the 100 % coverage gate from `pytest.ini`. `ci_success.png` shows all four jobs green.